Habib University
shaping futures

CS201 – Data Structures II

Lecture 15

Dr. Muhammad Mobeen Movania

Outline

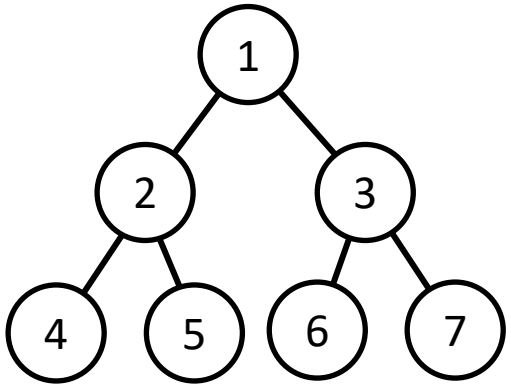
- What is a Heap?
- Types of Heap
- Two implementations of Heap
  - Heap as an Array (BinaryHeap)
  - Heap as a binary tree (MeldableHeap)
- Index calculation in BinaryHeap
- Merge operation in MeldableHeap
- Operations on heap
  - Add
  - Remove
- Amortized analysis
- Summary

What is a Heap?

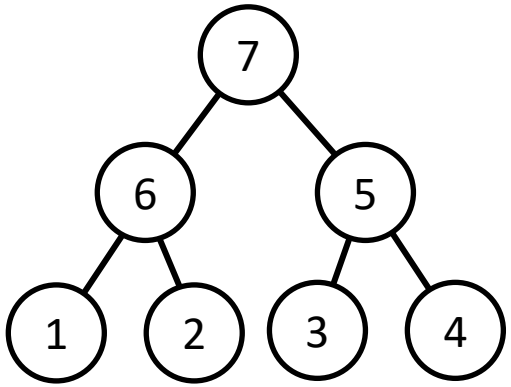
- Heap means a disorganized pile
- A special kind of a complete binary tree
- Its an implementation of a priority queue
- Two heap implementations exist
  - Based on an array (aka BinaryHeap)
  - Based on a binary tree with a meld (merge) operation (aka MeldableHeap)

Types of Heap

- Two types
  - Min heap: Root node is the min of all the children
  - Max heap: Root node is the max of all the children
- Heap property
  - For max heap, root is the largest node and for min heap, root is the smallest node of all the nodes



Min Heap



Max Heap

Exercise

Construct a max heap

Dataset: {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}

Exercise

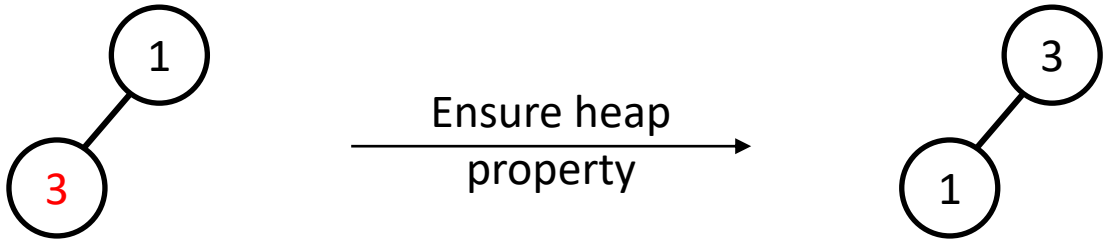
Construct a max heap

Dataset: {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}

Exercise

Construct a max heap

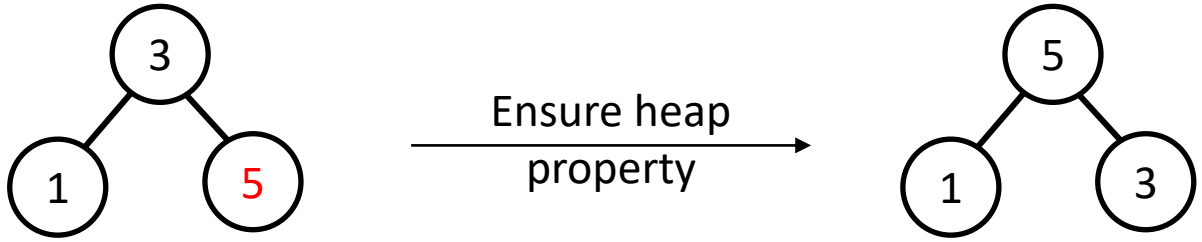
Dataset: {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}



Exercise

Construct a max heap

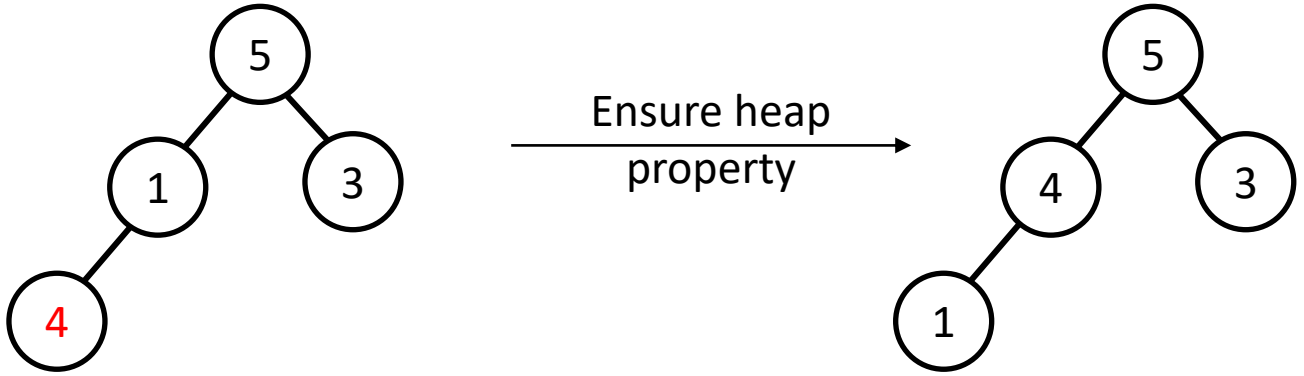
Dataset: {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}



Exercise

Construct a max heap

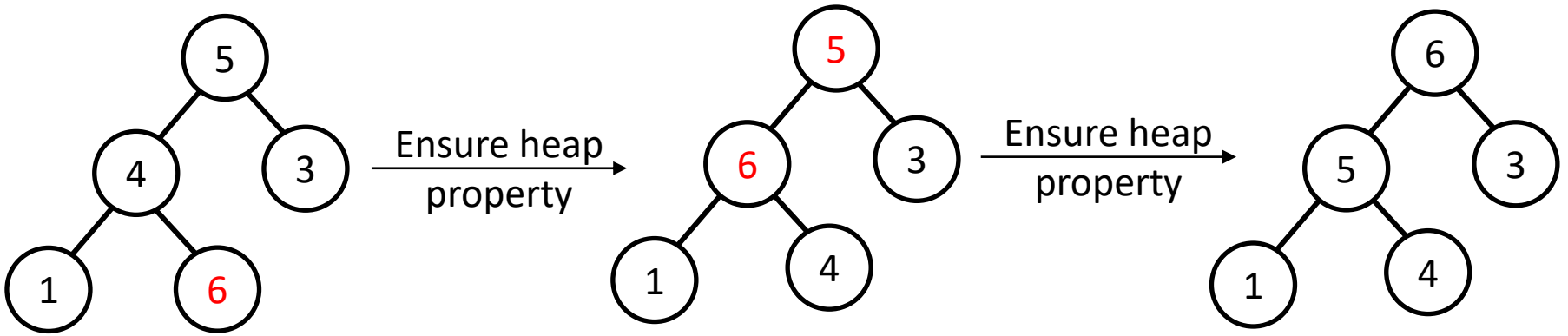
`Dataset: {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}`



Exercise

Construct a max heap

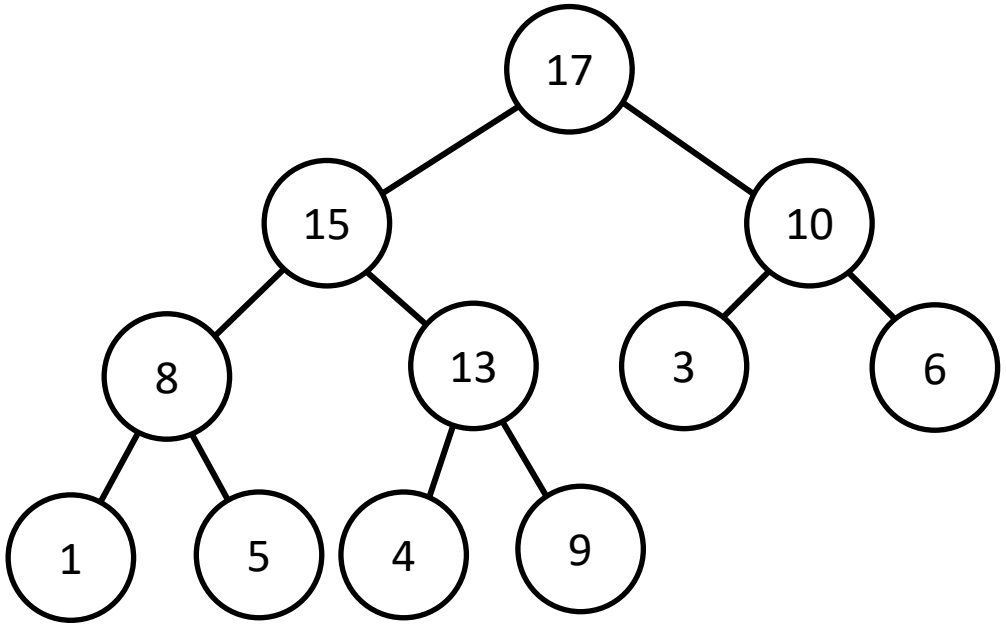
Dataset: {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}



Exercise

Construct a max heap

`Dataset: {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}`



Heap as an array

- We may store a complete binary tree as an array using Eytzinger's method

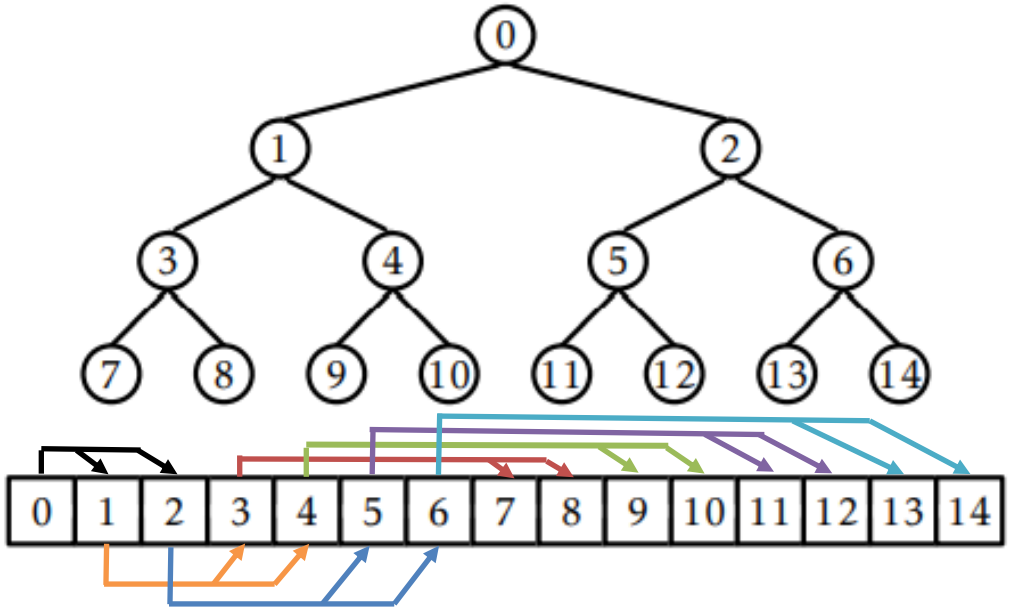


Figure 10.1: Eytzinger's method represents a complete binary tree as an array.

Heap as an array

- Root node has index 0
- Left child of root node has index 1
- Right child of root node has index 2

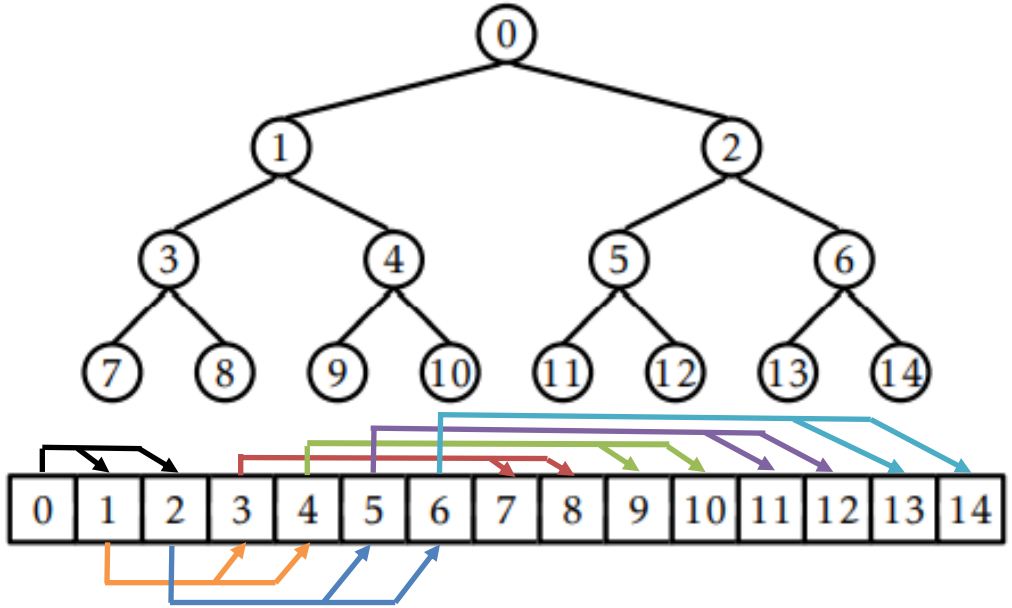


Figure 10.1: Eytzinger's method represents a complete binary tree as an array.

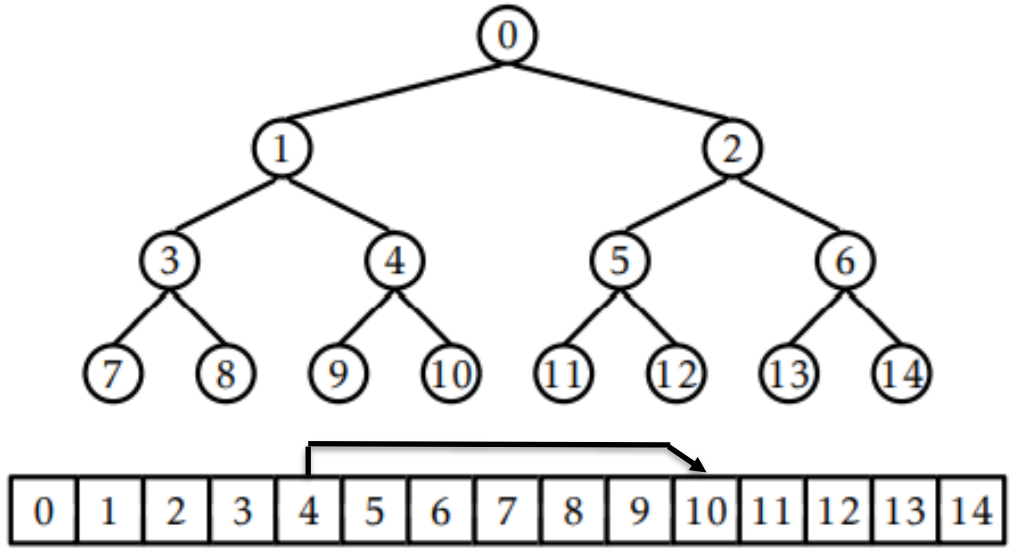
Index Calculation

Node	Level	index
0 (root)	0	0
1 (root->left)	1	1
2 (root->right)	1	2
3 (root->left)->left	2	3
4 (root->left)->right	2	4
5 (root->right)->left	2	5
6 (root->right)->right	2	6
…	…	…

Left child of a node at index i = 2*i + 1
Right child of a node at index i = 2*i + 2
Parent of a node at index i = (i -1)/2

Example

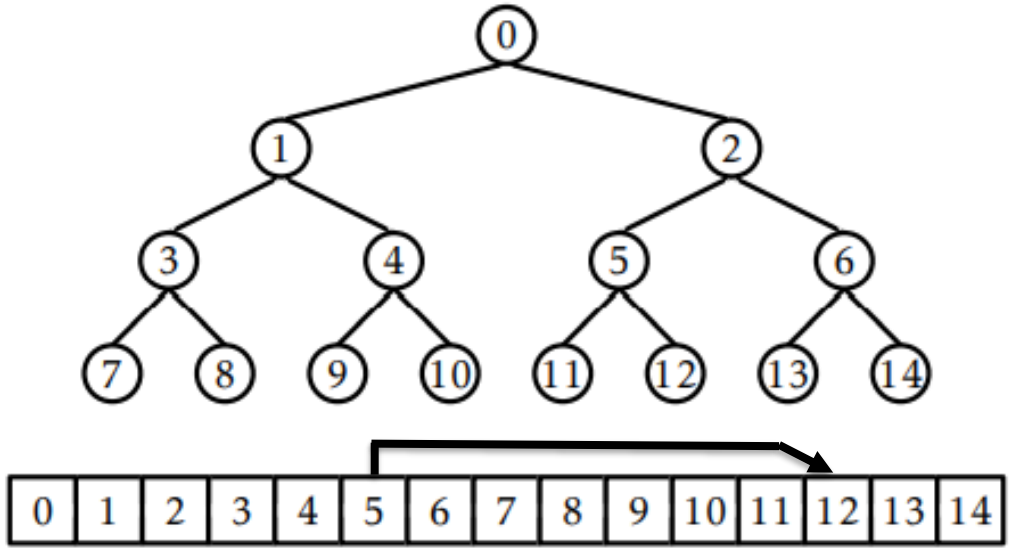
- Calculate index of parent of node 10



- Parent = (10-1)/2 = 9/2=4

Example

- Calculate index of right child of node 5



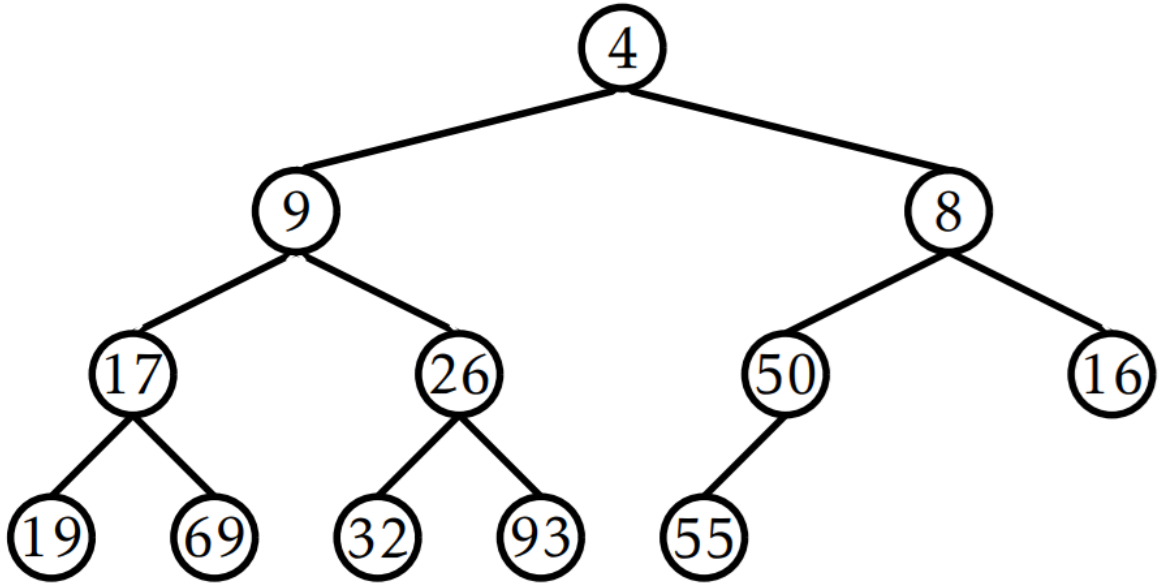
- Right child(5) = 2*5+2 = 12

Operations on BinaryHeap

- add operation
  - First check for sufficient space
  - Insert the new element at the end of array
  - Ensure heap property is maintained

Example

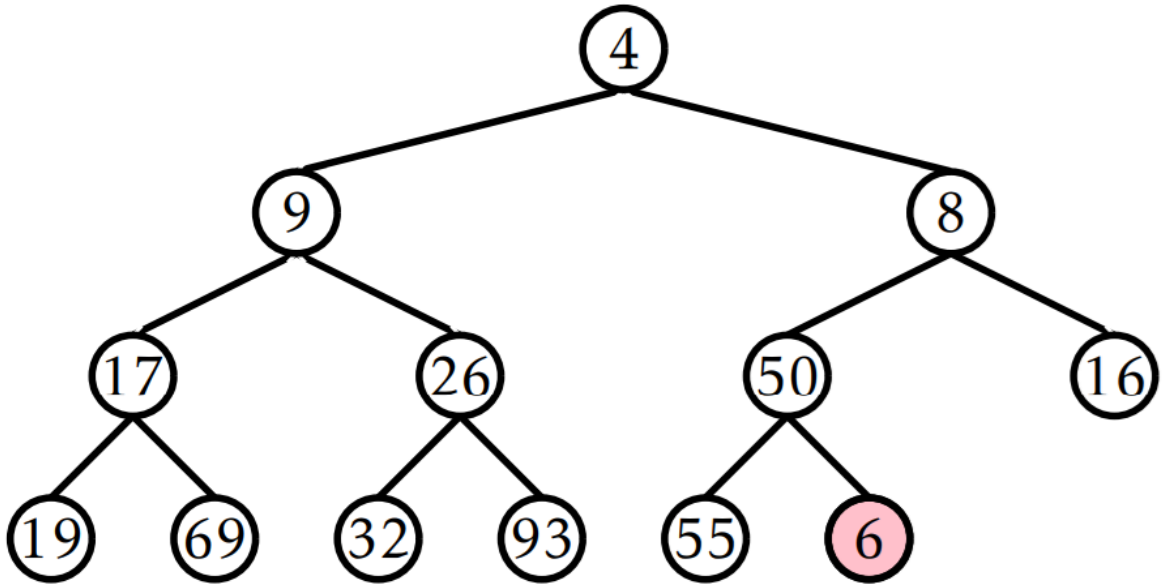
- add(6)
  - Check for sufficient space (Yes)

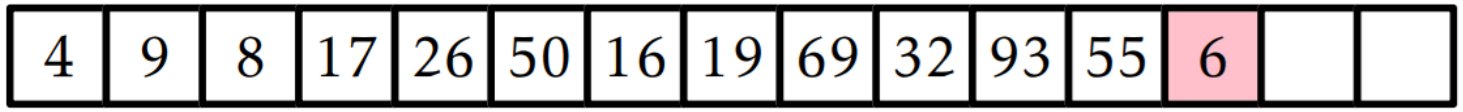


4	9	8	17	26	50	16	19	69	32	93	55			

Example

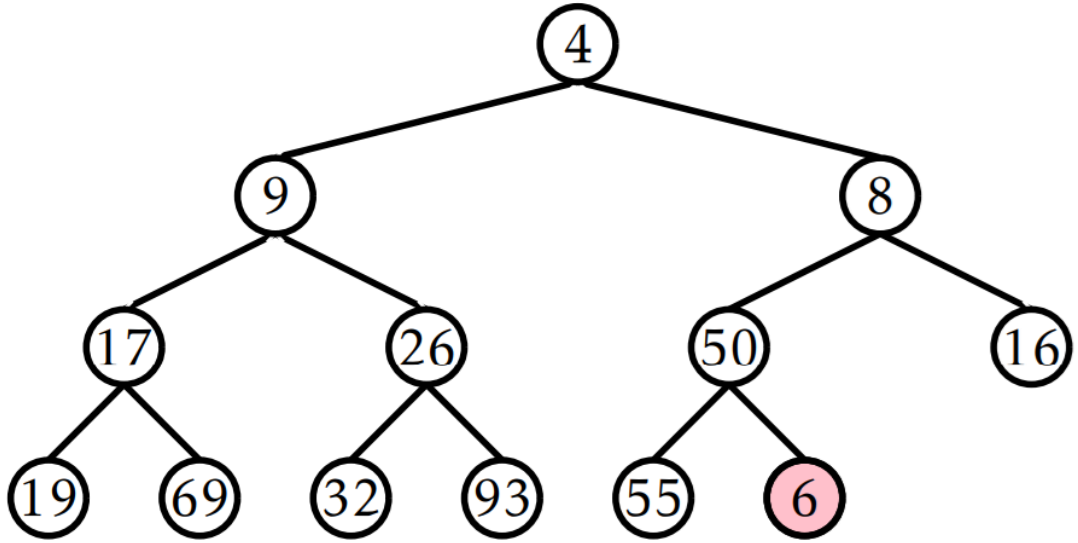
- add(6)
  - Insert 6 at the end of the array





Example

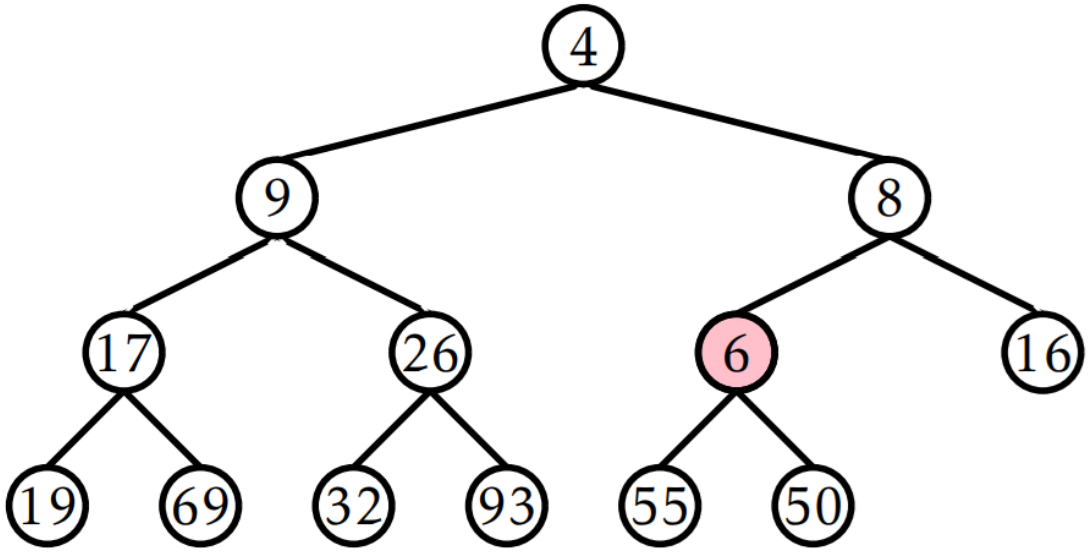
- add(6)
  - Get parent of 6 = (12-1)/2 =5



4	9	8	17	26	50	16	19	69	32	93	55	6		
0	1	2	3	4	**5**	6	7	8	9	10	11	**12**		

Example

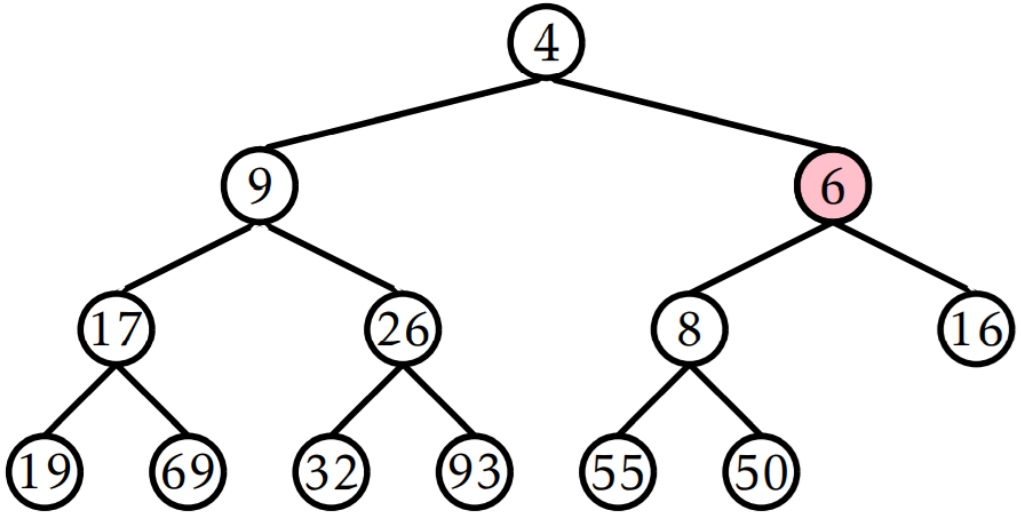
- add(6)
  - Ensure heap property (50>6) so we swap the two elements

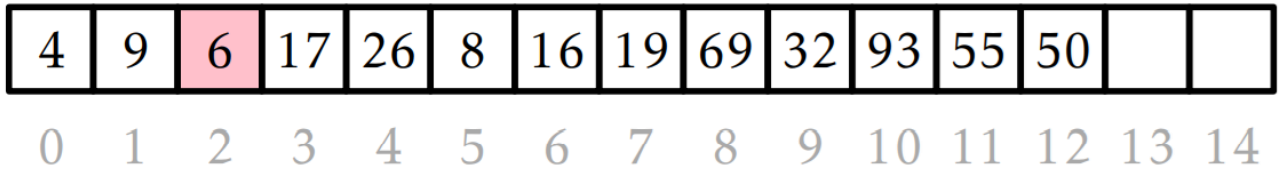


4	9	8	17	26	6	16	19	69	32	93	55	50		
0	1	2	3	4	**5**	6	7	8	9	10	11	**12**		

Example

- add(6)
  - Ensure heap property, swap elements, as 8>6





Pseudocode (add)

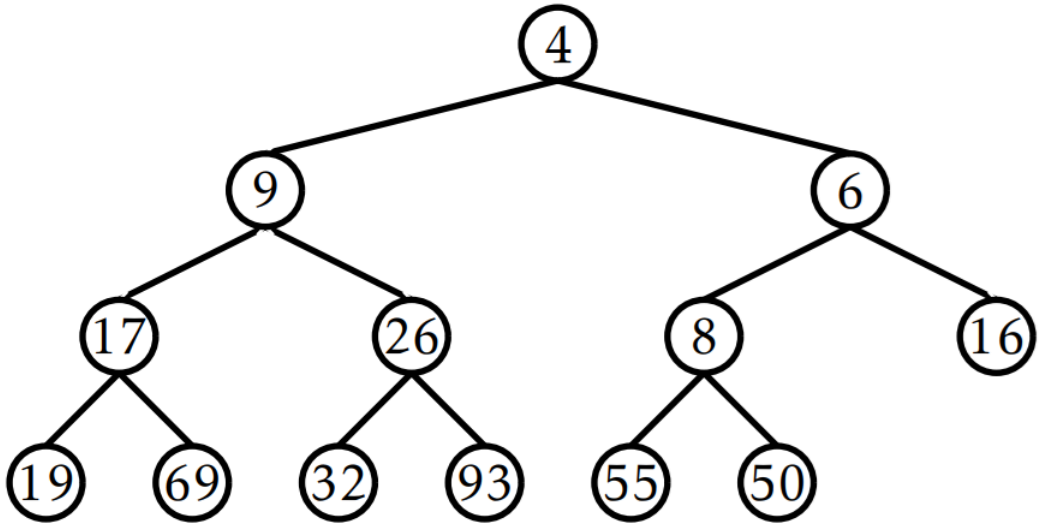
```
bool add(T x) {
  if (n + 1 > a.length) resize();
  a[n++] = x;
  bubbleUp(n-1);
  return true;
}
void bubbleUp(int i) {
  int p = parent(i);
  while (i > 0 && compare(a[i], a[p]) < 0) {
    a.swap(i,p);
    i = p;
    p = parent(i);
  }
}
```

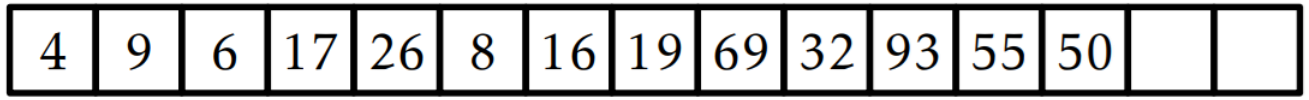
Operations on BinaryHeap

- remove()
  - Remove the smallest element from the heap which is the root node
  - Make the last element as the current root node
  - Ensure heap property by moving the new root node to its logical position

Example

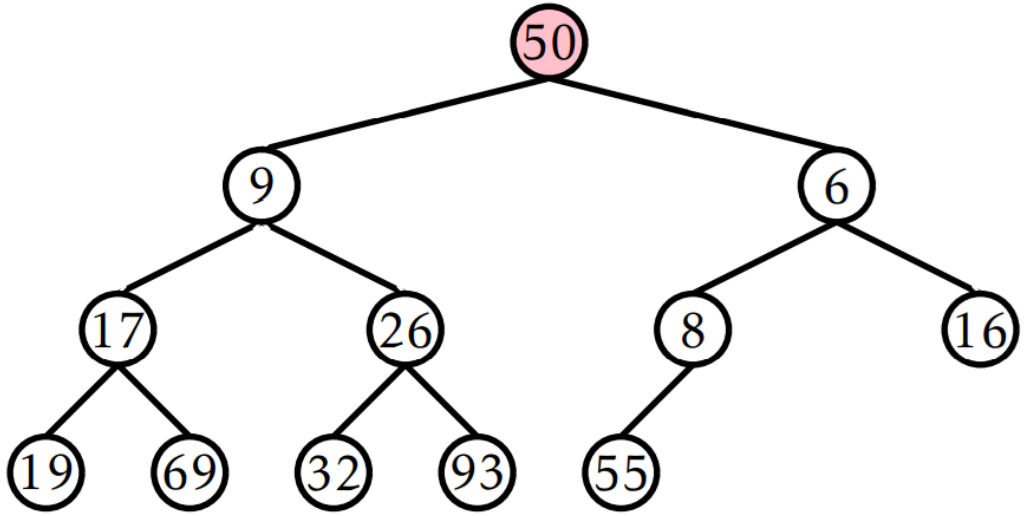
- remove()
  - Delete the current root (4) and replace with the last element (50)





Example

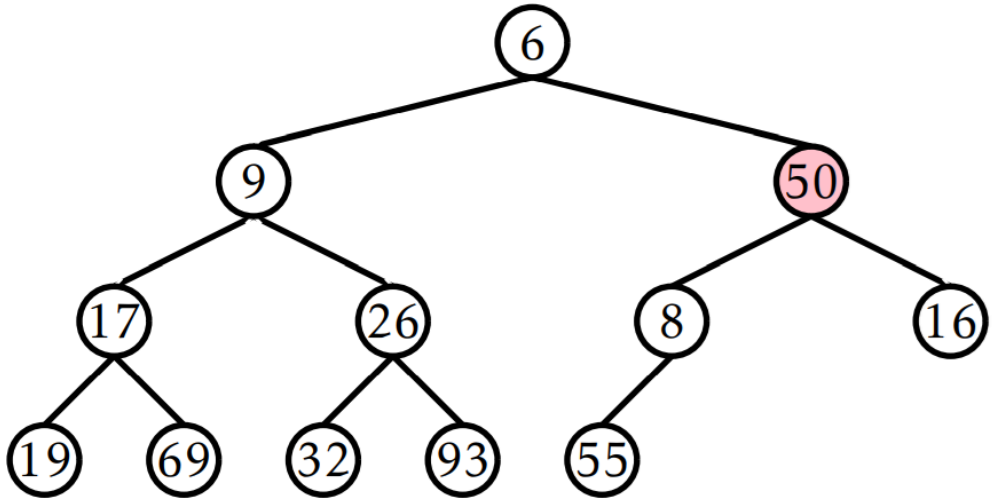
- remove()
  - Delete the current root (4) and replace with the last element (50)



50	9	6	17	26	8	16	19	69	32	93	55			

Example

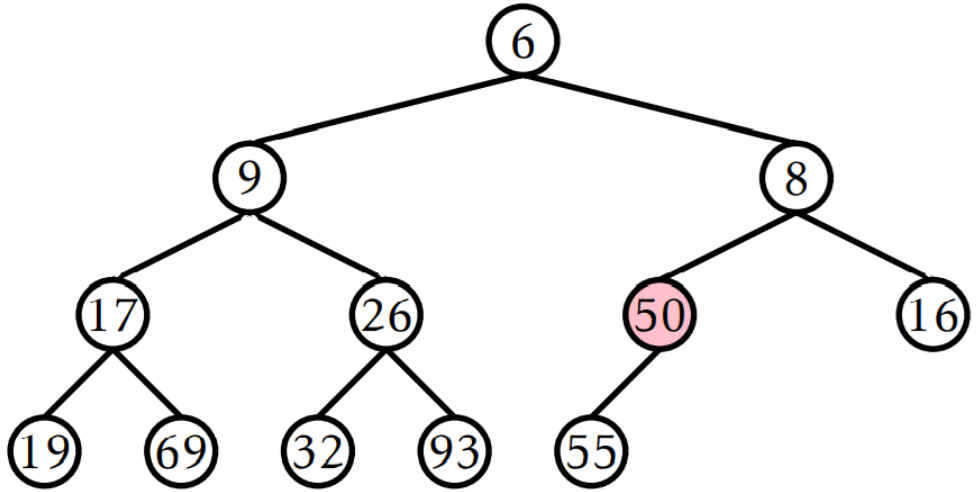
- remove()
  - Move the root element (50) down until heap property is not satisfied

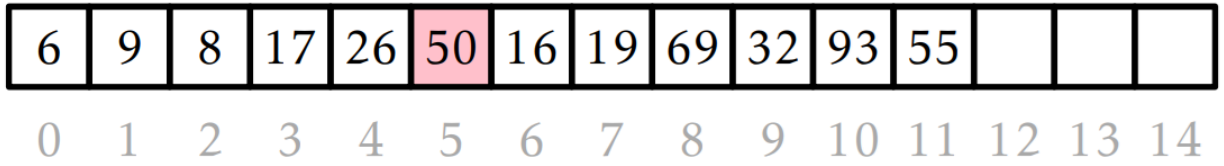


6	9	50	17	26	8	16	19	69	32	93	55			

Example

- remove()
  - Move the root element (50) down until heap property is not satisfied





Pseudocode (remove)

```
T remove() {
  T x = a[0];
  a[0] = a[--n];
  trickleDown(0);
  if (3*n < a.length) resize();
  return x;
}
```

- For details of trickleDown function, refer to the book chapter 10 page 215

Amortized Analysis (add and remove)

- The running time of both add and remove operations is dependent on the height of the heap
- Since heap is a complete binary tree, it will have the maximum possible no. of nodes for a given level

$$n >= 2^h$$

$$h <= \log n$$

- This gives the running time O(log(n))

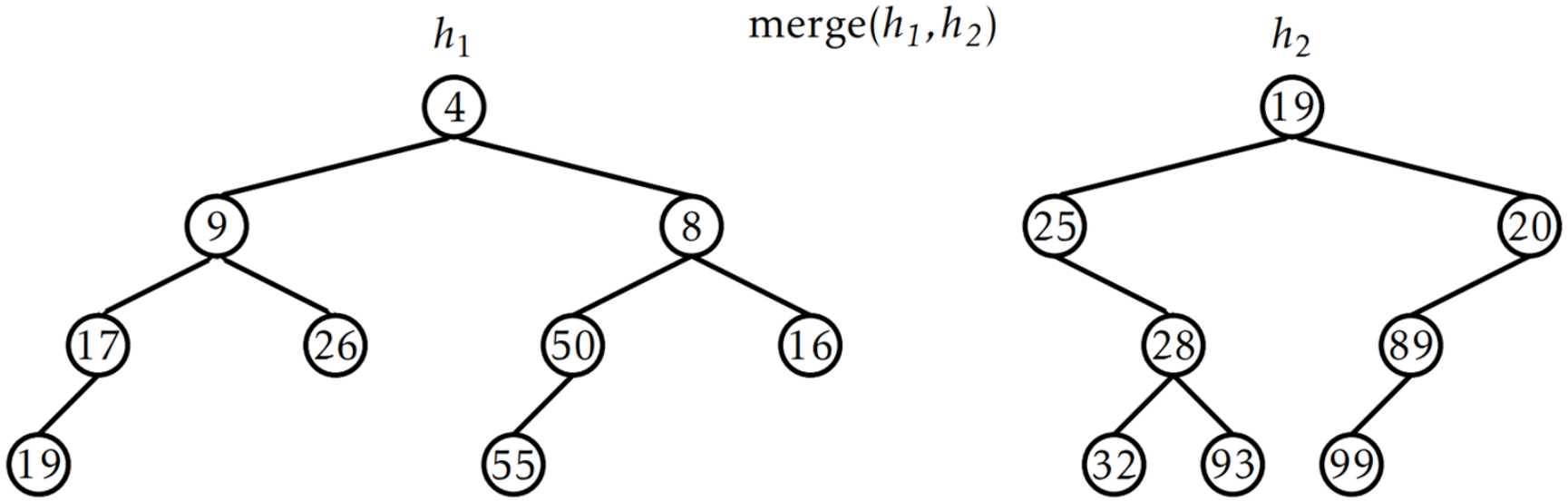
Meldable Heap

- It's a priority queue implementation based on a heap ordered binary tree
- No restrictions on the shape of the underlying binary tree
- The operations (add and remove) are implemented using merge(h_1, h_2) recursive operation
  - h_1 and h_2 are two heap nodes that are themselves root nodes of left and right subtrees

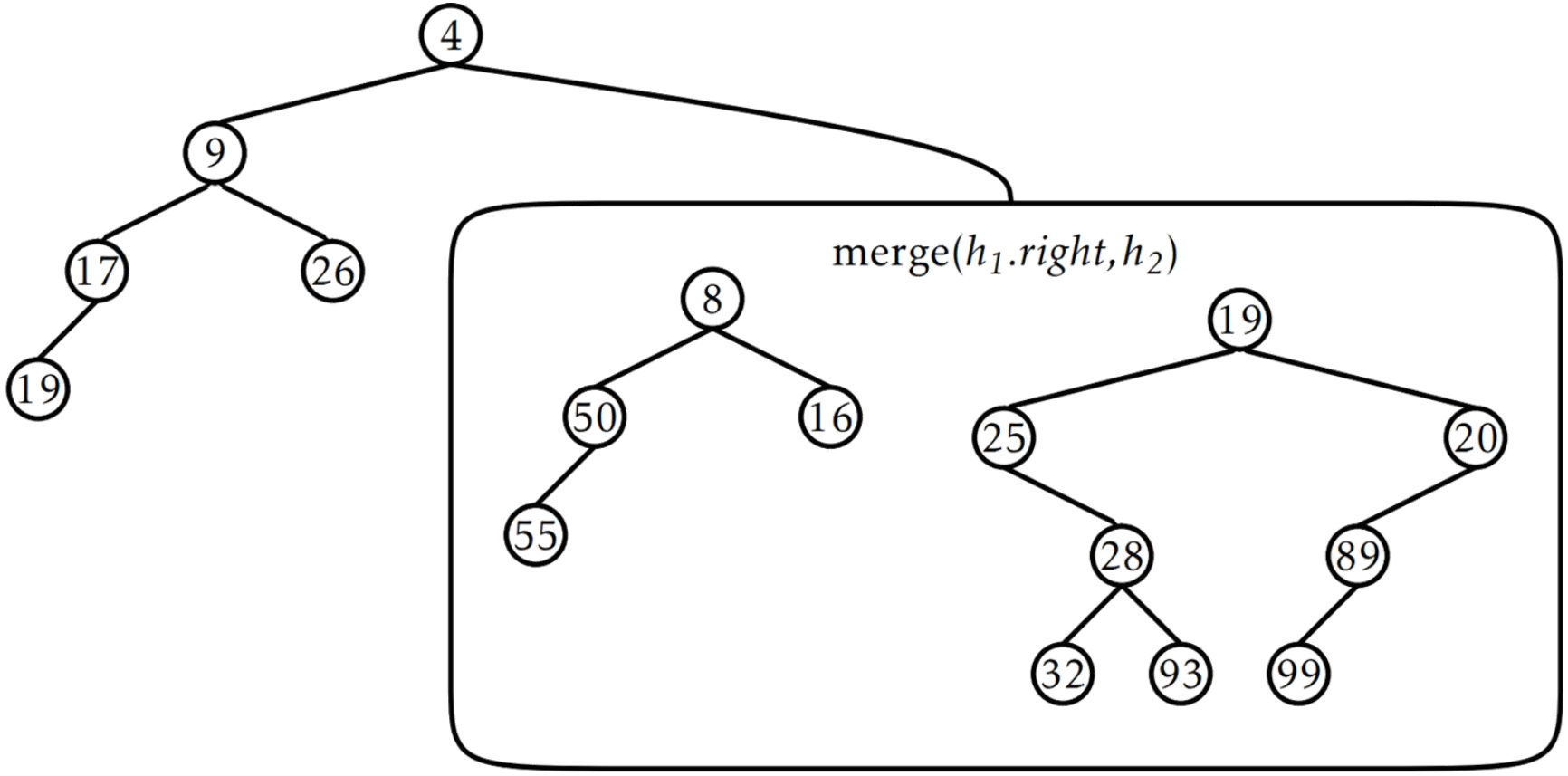
merge operation

- A useful recursive operation that helps in combining sub-heaps
- Merge operations takes the following steps
  - Check if h_1 or h_2 is nil, if so we return h2 or h1 respectively (since we are merging with a null set)
  - If $h_2 < h_1$, we reverse their roles (this ensures that the h_1 is smaller than h_2)
  - Root of merged node will have h_1.x so we recursively merge h_2 with h_1.left or h_1.right based on a random dice of coin.
- Expected time: O(log(n))

Example



Example



Pseudocode (merge)

```
merge(h1, h2)
    if h1 = nil then return h2
    if h2 = nil then return h1
    if h2.x < h1.x then (h1, h2) ← (h2, h1)
    if random_bit() then
        h1.left ← merge(h1.left, h2)
        h1.left.parent ← h1
    else
        h1.right ← merge(h1.right, h2)
        h1.right.parent ← h1
    return h1
```

Add operation (MeldableHeap)

- Add operation uses the merge operation as follows
  - Create a new node (u) to be added to the heap
  - Merge u with the root of heap (r) i.e. merge(u, r)
  - Increment the total number of nodes (n)
- Expected time: O(log(n))

Pseudocode (add)

```
add(x)
    u ← new_node(x)
    r ← merge(u, r)
    r.parent ← nil
    n ← n + 1
    return true
```

Remove operation (MeldableHeap)

- Similar to the add operation and easy to do with the merge operation
- Remove without any parameters, removes the root node,
  - Merge its two children and make the result the new root.
- A variant of remove that takes a parameter u (remove(u)) which removes the given node u from the heap
- Expected time: O(log(n))

Pseudocode (remove)

```
remove()
    x ← r.x
    r ← merge(r.left, r.right)
    if r ≠ nil then r.parent ← nil
    n ← n − 1
    return x
```

Summary

- We studies about heap and its types
- We saw two way of representing heaps, array based (BinaryHeap) and binary tree based (MeldableHeap)
- We saw how array based representation simplifies access to individual nodes
- We saw how merge operation is implemented and how it makes implementing add and remove functions easy
- We also saw that all operations on heap take an expected runtime of O(log(n))

Book Reading

- ODS
  - Heaps, Chapter 10 pages: 211-223